DATA FUSION THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

1. What is the role of CDAP in Cloud Data Fusion?

Cloud Data Fusion is built on top of an open-source framework called CDAP, which stands for Cask Data Application Platform. The role of CDAP is to provide the underlying engine and framework that makes the pipelines work. It gives Data Fusion things like the plugin architecture, the pipeline orchestration, and the ability to connect to different sources and sinks.

So basically, Cloud Data Fusion is the managed service that runs on GCP, but under the hood it uses CDAP for the heavy lifting like designing pipelines, running them as Spark or MapReduce jobs, and handling the metadata. CDAP makes it flexible and extensible, and Data Fusion wraps it with a user-friendly UI, Google integrations, and managed infrastructure.

2. How is Cloud Data Fusion different from open-source CDAP?

The main difference is that Cloud Data Fusion is a fully managed, Google-provided service, while CDAP is open source that you would need to deploy and manage yourself.

If I use open-source CDAP, I'd have to set up the clusters, take care of upgrades, scaling, monitoring, and security. With Cloud Data Fusion, Google handles all that for me I just focus on building pipelines.

Another difference is integrations. Cloud Data Fusion has deep integration with GCP services like BigQuery, Cloud Storage, Pub/Sub, and Dataproc. It also adds enterprise features like role-based access control with IAM, built-in logging and monitoring, and easy scaling. Open-source CDAP gives flexibility, but it doesn't come with those managed features out of the box.

So I'd say Cloud Data Fusion = CDAP + Google's management + GCP integrations, which makes it much easier to use in production.

3. What are the main components of Data Fusion (Studio, Wrangler, Metadata, Lineage)?

Cloud Data Fusion is made up of different components that make it easier to design, transform, and manage pipelines:

- Studio: This is the visual interface where I can drag and drop different sources, transforms, and sinks to design a pipeline. It's low-code and very user-friendly.
- Wrangler: This is where I can clean and prepare the data interactively. I can look at a sample of the data, apply transformations like renaming columns, splitting strings, or changing types, and then automatically convert those steps into pipeline code.
- Metadata: Data Fusion automatically collects metadata about datasets and pipelines, like schema, data source, and other properties. This helps in understanding what data exists and how it's being used.
- Lineage: This shows how data flows from source to destination across pipelines. It helps in tracking the history of data, debugging issues, and also useful for compliance because I can prove where data came from and how it was transformed.

Together, these components give both a developer-friendly and an operations-friendly environment: Studio and Wrangler help me build pipelines, and Metadata and Lineage help me monitor and govern them.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. What execution environments are supported for running pipelines in Data Fusion?

Cloud Data Fusion supports multiple execution environments because different workloads have different needs. By default, pipelines in Data Fusion can run on:

- Dataproc: This is a managed Hadoop and Spark cluster. Batch pipelines usually run on Dataproc as Spark jobs. Data Fusion automatically spins up the cluster, runs the pipeline, and tears it down if it's ephemeral.
- Cloud Dataflow: For streaming pipelines or when I need scalability with Apache Beam, Data Fusion can run pipelines on Dataflow.
- Local engine: There's also a preview or local mode used for quick testing, but that's not meant for production.

So, the choice of environment depends on whether the pipeline is batch or streaming. Batch pipelines often go to Dataproc (Spark), while streaming pipelines usually run on Dataflow.

5. How does Data Fusion integrate with other GCP services like BigQuery and Pub/Sub?

One of the biggest strengths of Data Fusion is its native integration with other GCP services.

- With BigQuery, I can directly use it as both a source and a sink in pipelines. For example, I can read from BigQuery tables or load data into BigQuery tables at the end of my pipeline. It's very seamless and supports both append and overwrite modes.
- With Pub/Sub, I can use it as a streaming source. Messages arriving in Pub/Sub can flow into a Data Fusion streaming pipeline, get transformed, and then be written to sinks like BigQuery, GCS, or even another Pub/Sub topic.
- It also connects well with GCS, Dataproc, Cloud Spanner, and other services. For example, I can land raw files in GCS, process them in a pipeline, and then store the result in BigQuery.

The key point is that I don't have to build custom connectors. Data Fusion provides built-in plugins for these services, so I can just drag them into my pipeline in Studio and configure them.

6. What is the difference between batch pipelines and streaming pipelines in Data Fusion?

The main difference is about how the data is processed.

- Batch pipelines work on finite datasets. For example, if I have a CSV file in GCS or a daily partition of data, a batch pipeline reads that whole dataset, transforms it, and writes it out to a target like BigQuery. It's good for scheduled, bulk processing.
- Streaming pipelines work on unbounded, continuous data. For example, if I'm consuming real-time events from Pub/Sub, a streaming pipeline will process the data as it arrives and push it to BigQuery or another sink almost instantly.

Under the hood, batch pipelines usually run on Spark via Dataproc, while streaming pipelines run on Dataflow using Apache Beam.

So, the choice depends on the use case. If I need real-time analytics or fraud detection, I'd go with streaming. If I'm loading a daily file dump, batch is the right choice.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. Give an example of a use case where batch pipelines are more suitable than streaming.

A very common example is daily reporting or end-of-day aggregation. Let's say a retail company collects transaction logs throughout the day. At midnight, they want to generate a full sales report, clean the data, and load it into a BigQuery warehouse. In this case, a batch pipeline makes more sense because the dataset is finite, and I only need to process it once per day.

Another example is when I receive bulk files from a third party like partner data or monthly CSV exports. That data naturally comes in chunks, not as a continuous stream, so batch pipelines are simpler and more cost-effective than keeping a streaming job running 24/7.

8. How does Data Fusion handle late-arriving or out-of-order data in streaming pipelines?

In streaming pipelines, late or out-of-order events are always a challenge. Data Fusion handles this by leveraging the underlying Dataflow (Apache Beam) capabilities. When I build a streaming pipeline, I can configure event-time processing with watermarks. A watermark tells the pipeline how long to wait for late data before considering a window of events as complete.

For example, if I expect events to sometimes be delayed by 5 minutes, I can set the watermark accordingly. Data Fusion will then hold the window open for that period and still process late-arriving events in the correct window. After the watermark passes, very late data is usually sent to a dead-letter path or logged separately.

So the way Data Fusion handles it is through Dataflow's event-time semantics, windowing, and triggers. As a user, I need to design my pipeline with the right tolerance for lateness depending on the business case.

9. What are the trade-offs between designing batch and streaming pipelines in Data Fusion?

The trade-offs usually come down to cost, complexity, and freshness of data.

- Batch pipelines are simpler and cheaper. They are great when data is not time-sensitive, and you only need periodic updates. But the trade-off is latency you don't see the data until the batch runs, which could be hours later.
- Streaming pipelines give near real-time data, which is very valuable for use cases like fraud detection, clickstream analysis, or IoT monitoring. The trade-off here is cost, because streaming jobs run continuously, and complexity, because I need to handle things like late data, windowing, and backpressure.

So, the decision is really about the business requirement. If the business needs fresh insights within seconds or minutes, streaming is worth the extra cost and complexity. If not, batch is simpler and more efficient.

10. What is a micro-batch pipeline and when would you use it?

A micro-batch pipeline is like a middle ground between batch and streaming. Instead of processing all data once a day like batch, or continuously like streaming, it processes data in very small batches at frequent intervals for example, every minute or every five minutes.

I would use micro-batch when true real-time streaming is not required, but waiting for a big daily batch is too slow. For example, if a company wants near-real-time sales dashboards but can tolerate a few minutes of delay, a micro-batch pipeline is perfect. It gives fresher data without the full cost and complexity of a continuously running streaming job.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. How do you build pipelines using the Studio in Data Fusion?

In Data Fusion, Studio is the visual interface where I design pipelines without writing much code. The way I usually explain it is like this:

When I open Studio, I get a canvas where I can drag and drop different components. These components represent sources (like BigQuery, GCS, Pub/Sub), transformations (like Wrangler, join, filter, aggregate), and sinks (like BigQuery, GCS, Pub/Sub). I connect them visually to show the flow of data.

For example, if I want to load a CSV file from GCS, clean it, and put it into BigQuery, I would drag the GCS source, add a Wrangler transform, and connect it to a BigQuery sink. Once the design is ready, Studio automatically generates the pipeline config behind the scenes.

I also configure properties for each node like table name, schema, file path directly in the UI. Then I can validate the pipeline, preview it with sample data, and save it. It's very intuitive, and I don't need to manage Spark or Dataflow code manually.

12. What are the steps involved in deploying and running a pipeline?

Once I finish designing a pipeline in Studio, the next steps are:

1. Validate the pipeline – check for errors or missing configurations.
2. Save and deploy – when I click deploy, Data Fusion packages the pipeline definition and makes it ready to run.
3. Run the pipeline – I can trigger it manually, on-demand, by clicking run, or I can set parameters at runtime if the pipeline is parameterized (like passing in file paths).
4. Execution environment – behind the scenes, Data Fusion submits the pipeline to the right engine, like Dataproc for Spark batch jobs or Dataflow for streaming jobs.
5. Monitor – I can monitor execution in the Data Fusion UI. It shows logs, metrics, and the current status of each stage. If there's an error, I can look at the logs or lineage to debug.

So in short: design → validate → deploy → run → monitor.

13. How can you schedule a pipeline to run automatically in Data Fusion?

Data Fusion lets me schedule pipelines using built-in scheduling options. After I deploy a pipeline, I can go to the schedule tab and set up how I want it to run.

There are two common ways:

- Time-based scheduling – like cron jobs. For example, I can run a pipeline every day at midnight, or every 15 minutes.
- Event-based scheduling – for example, a pipeline starts when a file lands in a GCS bucket, or when a Pub/Sub message arrives.

If I need more flexibility, I can also use Cloud Scheduler + Pub/Sub to trigger pipelines indirectly. But most of the time, the built-in scheduling inside Data Fusion is enough.

So if the requirement is daily loads into BigQuery, I just configure a daily schedule in the UI, and Data Fusion handles the automatic runs for me.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. How do you monitor pipeline execution and logs in Data Fusion?

When I run a pipeline in Data Fusion, I can monitor it directly in the Data Fusion UI. In the monitoring section, I can see the pipeline's status whether it's running, succeeded, or failed. It also shows me a visual of each stage, so I know which part of the pipeline is currently executing and how much data has passed through each stage.

For deeper details, Data Fusion integrates with Cloud Logging. That means every pipeline execution automatically writes logs into GCP's logging service. From there, I can filter logs by pipeline name, stage, or even error type. This is very useful when I need to troubleshoot.

Data Fusion also provides metrics like number of records processed, time taken per stage, and overall pipeline duration. For long-running streaming jobs, I can check real-time metrics to see if data is flowing as expected.

So in short, I use the Data Fusion UI for quick monitoring and Cloud Logging for detailed investigation. I can even set up alerts in Cloud Monitoring if certain error logs or failure metrics appear too often.

15. How does Cloud Composer integrate with Data Fusion for orchestration?

Cloud Composer is great for orchestrating workflows that involve multiple steps or systems, and Data Fusion can be one of those steps. The integration usually happens through the Data Fusion REST API.

In practice, what I would do is: I create an Airflow DAG in Cloud Composer that calls the Data Fusion API to start a pipeline. I can pass in parameters like which pipeline to run, runtime arguments, or even which instance to use. After triggering, the DAG can also check the status of the pipeline until it completes.

This way, I can build workflows where, for example, first a Data Fusion pipeline ingests and cleans raw data, then a BigQuery query runs to aggregate it, and finally another step sends a notification. Composer ensures these steps run in order and handles retries or dependencies.

So the key point is Composer doesn't run the Data Fusion pipeline itself, it just orchestrates by calling its APIs. That makes it very useful when pipelines are part of a bigger end-to-end process.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16. What are some common source plugins available in Data Fusion?

Data Fusion comes with a rich set of source plugins, which are basically connectors that let me read data from different systems. Some of the most commonly used ones are:

- Cloud Storage (GCS) – very common for ingesting CSV, JSON, Avro, Parquet files. I can easily point to a bucket or prefix.
- BigQuery – I can use BigQuery tables directly as a source. Useful when I want to feed one table into a pipeline for further processing.
- Pub/Sub – used for real-time streaming sources. Events arriving in Pub/Sub can flow into a streaming pipeline.
- Databases (JDBC) – connectors for MySQL, PostgreSQL, SQL Server, Oracle, etc. These allow me to pull data from transactional databases into pipelines.
- Spanner – a source connector for Cloud Spanner, which is Google's distributed relational database.
- Salesforce – many enterprises use Salesforce as a CRM, and Data Fusion has a plugin to extract that data.
- Kafka – another streaming source that can be used if the organization runs Kafka clusters.
- FTP/SFTP – useful for legacy systems where data files are dropped into servers.

Each plugin lets me configure connection details, authentication, and query or path options. Once configured, I can use them directly in Studio by dragging the source into the canvas.

In real projects, I've mostly used GCS, BigQuery, and Pub/Sub sources because they fit very naturally in GCP pipelines, but having database and SaaS connectors available makes Data Fusion flexible for hybrid data environments.

17. Name a few built-in transformation plugins in Data Fusion.

Data Fusion has several built-in transformation plugins that let me clean, enrich, or reshape data without writing custom code. A few common ones are:

- Wrangler – for interactive data cleaning, like renaming columns, splitting values, changing types.
- Deduplicate – removes duplicate records based on a key field.
- Joiner – allows me to join two datasets together, similar to SQL joins.
- GroupBy / Aggregator – lets me apply aggregate functions like sum, avg, count, grouped by a field.
- Filter – filters out records based on conditions.
- Normalizer / Denormalizer – reshape data between wide and tall formats.
- Parser – parses data from formats like JSON or XML into structured records.
- Projection – select or reorder specific fields.

These are all drag-and-drop in the Studio, and I can chain them together to build complex transformations quickly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. Which sink plugins are most commonly used for writing pipeline outputs?

The most commonly used sink plugins are the ones that connect to GCP's storage and analytics services:

- BigQuery sink – probably the most popular. I can write the transformed data into BigQuery tables for analytics. It supports modes like append or overwrite.
- Cloud Storage (GCS) sink – used when I need to store output as files, for example in CSV, JSON, Avro, or Parquet format.
- Pub/Sub sink – useful for event-driven architectures, where the pipeline output needs to be published as a stream of messages.
- Databases (JDBC sinks) – for writing results back to operational databases like MySQL, PostgreSQL, or SQL Server.
- Spanner sink – when writing back into Cloud Spanner tables.
- Elasticsearch sink – for search and analytics use cases.

In practice, most of my projects have used BigQuery as the final sink, since that's where reporting and analytics happen. GCS is also common for archiving processed data.

19. How can you develop a custom plugin for Data Fusion?

If the built-in plugins don't cover my use case, I can build a custom plugin. The process is:

1. Write the plugin in Java using the CDAP API, since Data Fusion is built on CDAP. I define the logic for either a source, transform, or sink.
2. Package the plugin into a JAR file.
3. Deploy the plugin into the Data Fusion instance by uploading it through the Hub or Admin page.
4. Once deployed, it shows up in Studio just like any other plugin, and I can drag and use it.

For example, if I need to connect to a niche API that doesn't have a connector, I'd build a custom source plugin for it. Or if I want a very specific transformation, I can implement it as a custom transform.

20. How do you manage the lifecycle of plugins in Data Fusion?

Managing plugin lifecycle means handling how plugins are added, updated, and retired. In Data Fusion, plugins are managed at the instance level.

- **Deployment** – I upload the plugin JAR into the instance. It becomes available to all pipelines in that instance.
- **Versioning** – I can upload multiple versions of a plugin. Pipelines that use an older version continue to work, while new pipelines can choose the newer one.
- **Updates** – if I need to upgrade, I usually test the new version in a lower environment (like dev) before deploying to prod.
- **Deletion/retirement** – unused plugins can be removed, but I need to make sure no active pipelines depend on them.
- **Governance** – usually teams maintain plugins in a source control repo, build them with CI/CD, and deploy them consistently across environments to avoid drift.

© Shubham Wadekar

So, in practice, I treat plugins almost like software packages: I version them, test them, and control their deployment carefully so pipelines remain stable.

21. What is the role of Wrangler in Data Fusion?

Wrangler is the interactive data preparation tool inside Data Fusion. Its main role is to let me quickly explore and clean raw data before putting it into a pipeline. When I open Wrangler, I can load a sample of the dataset for example, a CSV from GCS or a table from BigQuery and then apply transformations step by step in a no-code way.

Each step I apply, like renaming a column or changing a type, gets recorded as a "directive." Once I'm satisfied with the cleaned data, Wrangler can automatically generate a transform node for the pipeline with all those steps included.

So the value of Wrangler is that I don't need to write complex code to clean data. I can experiment interactively, then directly push those cleaning steps into a production pipeline. It saves time and reduces errors compared to manual coding.

22. Give an example of a wrangler directive you can use to clean data.

One simple example is the split directive. Let's say I have a column with full names like "John Smith." Using Wrangler, I can apply split col-name ' ' which will split the column into two new columns first name and last name.

Another common one is drop, which removes unnecessary columns. For example, drop col-address would delete the address column if I don't need it.

I've also used parse-as-date to convert a string column like "2025-09-11" into an actual date type, which makes it easier to use later in BigQuery.

So directives are like small, scriptable instructions, but Wrangler makes them point-and-click friendly.

23. How can you perform joins between datasets in a Data Fusion pipeline?

In Data Fusion, joins are done using the Joiner plugin. In the Studio, I add the Joiner transformation and connect it to two or more sources. Inside the Joiner configuration, I specify the join keys and the type of join inner join, left outer join, right outer join, or full outer join.

For example, if I have customer details in one dataset and transactions in another, I can join them on customer_id. The Joiner then produces a single unified dataset that flows into the next stage of the pipeline.

The nice thing is it's very similar to how SQL joins work, but I do it visually. The Joiner also lets me select which fields I want to keep from each side of the join, so the output schema stays clean.

For very large datasets, I'd be careful about performance, but for most pipelines, the Joiner works smoothly and gives me relational-style functionality inside Data Fusion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. What transformation would you use to remove duplicate records?

In Data Fusion, I would use the Deduplicate transformation plugin. It allows me to define one or more fields as the key, and the plugin will remove rows that have the same key values. For example, if I have customer data and some records repeat with the same customer_id, I can configure Deduplicate to keep only one record for each unique customer.

I can also control which record to keep like the first occurrence, the last occurrence, or based on a specific field (for example, keep the most recent timestamp). This helps in cleaning up raw datasets before loading them into BigQuery or any downstream system.

25. Why should you consider push-down transformations instead of doing everything in the pipeline?

Push-down transformations mean letting the underlying system (like BigQuery, Dataproc, or database engines) do part of the processing, instead of Data Fusion doing all the heavy lifting itself.

The advantage is performance and cost. For example, if I'm reading from BigQuery, instead of pulling the entire table into the pipeline and then filtering rows, I can push the filter condition down to BigQuery. BigQuery is optimized for that, so it will only return the relevant rows. This reduces the amount of data Data Fusion has to process and speeds up the pipeline.

It also saves money because I avoid unnecessary data transfer and compute usage in the pipeline. So the main reason I'd consider push-down is efficiency I let the storage/compute engine that's closest to the data do the heavy work.

26. How can you load data into a partitioned BigQuery table using Data Fusion?

To load data into a partitioned BigQuery table, I would use the **BigQuery sink plugin** in my Data Fusion pipeline. When I configure the sink, I can point it to a destination table that is already created in BigQuery with partitioning enabled.

Partitioning can be by ingestion time or by a column like date. If it's ingestion-time partitioning, the records will automatically go into the correct partition based on when they arrive. If it's column-based, I just need to make sure my input data has that column populated for example, a transaction_date field.

In the sink configuration, I can also choose write disposition, like append or overwrite, depending on how I want to handle data for partitions.

So in practice, the steps are:

1. Create the partitioned table in BigQuery ahead of time with the right partition column.
2. Build the pipeline in Data Fusion with transformations as needed.
3. Use the BigQuery sink and map the fields, ensuring the partition column is present.

This way, the data is loaded neatly into the right partitions, which makes querying more efficient later.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. What's the difference between streaming vs batch writes to BigQuery from Data Fusion?

The main difference is in how the data gets delivered into BigQuery.

With batch writes, Data Fusion collects the output of a pipeline and then loads it into BigQuery using a load job. This is efficient for large amounts of data because BigQuery can optimize the bulk load. It's cheaper and better suited for use cases like daily or hourly ingestions. The trade-off is latency the data appears in BigQuery only after the batch finishes.

With streaming writes, Data Fusion writes records into BigQuery in real time using the streaming insert API. This is useful when I need near real-time data, like fraud detection or live dashboards. The trade-off here is cost and quota limits streaming inserts are more expensive, and they have per-second row limits.

So, I'd say batch is better for high-volume, scheduled loads, while streaming is better when freshness is critical.

28. How is schema mapping handled between Data Fusion pipelines and BigQuery tables?

Schema mapping in Data Fusion is handled during the sink configuration. When I use the BigQuery sink plugin, I can define how fields in my pipeline map to columns in the BigQuery table.

If the field names and types match, it's straightforward Data Fusion automatically maps them. If not, I can use transformations in the pipeline (like Wrangler, projection, or type conversion) to adjust the schema before writing. For example, if my pipeline has a string date, I might convert it into a timestamp type so it matches the BigQuery column.

Data Fusion also lets me use a directive or transformation to rename fields so they align with BigQuery's schema. In practice, I always make sure the schema in the pipeline matches the BigQuery table before deploying, otherwise the load will fail.

29. What does "push-down" transformation to BigQuery mean in Data Fusion?

Push-down means that instead of Data Fusion doing the transformation itself, it sends that transformation logic down to BigQuery and lets BigQuery perform it.

For example, if my pipeline is reading from BigQuery and I apply a filter stage in Data Fusion, push-down optimization will actually rewrite that filter as part of the SQL query that runs on BigQuery. That way, only filtered rows are returned to the pipeline.

This is more efficient because BigQuery is designed for large-scale SQL processing, so it's faster and cheaper to let it handle filtering, aggregations, or projections. It also reduces data movement into the pipeline.

So in short, push-down transformations make the pipeline smarter by letting BigQuery do the work it's best at.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. How can Data Fusion handle schema evolution when writing into BigQuery?

Schema evolution is always tricky, but Data Fusion has a few ways to manage it.

If the source schema changes slightly for example, a new column is added I can update the pipeline to include that new column and then map it to the BigQuery table. BigQuery allows adding new nullable columns, so the pipeline will still succeed.

If the change is breaking like a column type changes I'd need to handle it with a transformation step to cast or standardize the field before loading. Sometimes I'll write into a staging table first, where schema can be more flexible, and then run a merge query in BigQuery to adjust the data into the final schema.

Another approach is to use Wrangler to dynamically adjust the schema during pipeline design. Data Fusion also supports schema propagation, which means changes in earlier stages can automatically flow downstream, but I usually still check the final mapping before writing.

In real projects, I've seen teams manage schema versions in a metadata store and update Data Fusion pipelines whenever the schema evolves. That way, schema evolution doesn't break production loads.

31. How can Data Fusion read events from a Pub/Sub topic?

Data Fusion has a built-in Pub/Sub source plugin that lets me connect directly to a Pub/Sub topic. When I configure it, I specify the project ID, the topic name, and the subscription details.

For streaming pipelines, this works very naturally: the pipeline continuously listens to the Pub/Sub topic, picks up new messages as they arrive, and passes them downstream for transformation and loading. For example, I can stream clickstream events from Pub/Sub into Data Fusion, clean them with Wrangler, and then write them into BigQuery in near real time.

If the messages are structured, like JSON or Avro, I can define the schema in Data Fusion so the fields are parsed into proper columns. If they're raw strings, I might need to add a parser transformation before moving forward.

So, with the Pub/Sub source plugin, Data Fusion becomes a streaming consumer of Pub/Sub events.

32. How can Data Fusion write data back into Pub/Sub?

For writing, Data Fusion provides a Pub/Sub sink plugin. I configure it with the project ID and topic name where I want to publish messages.

Any data that reaches the sink stage in the pipeline is converted into Pub/Sub messages and published. This is useful in cases like data enrichment or filtering. For example, I can consume events from one Pub/Sub topic, apply a transformation in Data Fusion, and then write the cleaned or enriched messages into another Pub/Sub topic for downstream consumers.

I can also choose whether to publish the entire record as JSON or map specific fields into the message payload and attributes. This flexibility helps when different systems downstream expect different formats.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. How is schema evolution managed when Pub/Sub messages change?

Schema evolution with Pub/Sub is a bit different from databases because Pub/Sub itself doesn't enforce schema it just sends messages, often JSON. That means the pipeline needs to handle changes gracefully.

In Data Fusion, I usually manage this by:

- Using schema registry or metadata: Some teams maintain a schema definition outside Pub/Sub, and the pipeline validates messages against it.
- Making fields nullable or optional: If new fields are added to the message, I can update the Data Fusion schema to include them as optional so older messages still parse.
- Transformations to handle missing fields: If a field is missing in some messages, I can use Wrangler or expression transformations to provide defaults.
- Backward compatibility: If the change is breaking, like a type change, I usually cast or normalize the field in Data Fusion before writing into BigQuery or another sink.

In real-world projects, I've seen schema evolution handled by versioning the messages. For example, each Pub/Sub message has a version field, and Data Fusion pipelines check the version and parse accordingly. That way, older consumers aren't broken when the schema changes.

34. What is the role of a dead-letter topic in streaming pipelines with Pub/Sub?

A dead-letter topic is like a safety net in a streaming pipeline. Its role is to catch messages that cannot be processed successfully after several retry attempts.

In a Pub/Sub → Data Fusion streaming pipeline, if a message is corrupted, doesn't match the expected schema, or keeps failing due to transformation errors, we don't want it stuck in the subscription backlog forever. Instead, Pub/Sub can route it to a dead-letter topic once it has exceeded the maximum delivery attempts.

This way, bad messages are isolated. I can monitor the dead-letter topic, inspect the failed events, and either fix the data or replay them later after correcting the issue. It prevents a single bad message from blocking the entire pipeline and gives visibility into data quality problems.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

35. How would you design a real-time Pub/Sub → Data Fusion → BigQuery pipeline?

To design this, I'd start with Pub/Sub as the entry point. Let's say events are being published from a web application or IoT device.

1. Pub/Sub source: In Data Fusion, I configure a streaming pipeline with the Pub/Sub source plugin pointing to that topic.
2. Transformations: In the pipeline, I add transformations like Wrangler to parse JSON, filters to remove unwanted events, and maybe a join with a reference dataset if enrichment is needed.
3. Schema mapping: I make sure the message fields are mapped properly to the target schema. If needed, I handle missing or malformed fields with defaults.
4. BigQuery sink: Finally, I configure the BigQuery sink plugin. Since this is real-time, I use streaming inserts into a BigQuery table. If the use case needs partitioning, I make sure the table is partitioned on a date/timestamp field.
5. Error handling: For invalid messages, I send them to a Pub/Sub dead-letter topic or write them into a GCS error bucket.
6. Monitoring: I use Data Fusion's monitoring and Cloud Logging to keep track of pipeline health, latency, and errors.

With this design, events flow continuously from Pub/Sub, get cleaned and enriched in Data Fusion, and land in BigQuery within seconds for real-time analytics.

36. What file formats from GCS are supported by Data Fusion ingestion?

Data Fusion supports multiple file formats when reading data from GCS:

- CSV – very common for tabular data, with options to handle headers, delimiters, and quoted fields.
- JSON – for semi-structured data, either newline-delimited or JSON arrays.
- Avro – a popular binary format, especially for big data pipelines.
- Parquet – another efficient columnar format often used in analytics.
- ORC – supported for some Hadoop-based use cases.
- Text – plain text files, which I can parse with additional transformations.

In practice, CSV and JSON are the most common formats I've seen in day-to-day pipelines, while Avro and Parquet are more common when working with big datasets that need efficient storage and query performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. How do you design a pipeline that ingests only new files from GCS?

If I want to make sure my pipeline only ingests new files, I usually design it in a way that tracks what has already been processed. The simplest method is to use GCS notifications to Pub/Sub whenever a new file is finalized in a bucket, it triggers the pipeline. That way, only new files are picked up.

Another approach I often use is the "processed folder" pattern. New files arrive in an incoming/ folder, and once a pipeline successfully processes them, I move them to processed/. This ensures that only unprocessed files are left in incoming/.

In more advanced cases, I've also used a metadata table every time a file is processed, I record its name and checksum in BigQuery. Before processing, the pipeline checks against this table to confirm it hasn't seen the file before. That gives a strong safeguard against duplicates.

38. How can you ingest data into partitioned BigQuery tables from GCS via Data Fusion?

For this, I usually first create the target table in BigQuery with partitioning enabled either ingestion-time partitioning or based on a column like transaction_date.

Then in Data Fusion, I set up a pipeline with GCS as the source and BigQuery as the sink. If I'm using ingestion-time partitioning, the data will automatically land in the right partition as it's loaded. If I'm using column-based partitioning, I make sure my source data has that partition column, so rows go into the right partitions.

This way, queries downstream can take advantage of partition pruning, which keeps costs lower and performance faster. I've found this pattern very effective for time-series data like logs or transactions.

39. What happens if the schema of incoming GCS files does not match the target table schema?

If the schema doesn't match, the pipeline will usually fail at the load step, because BigQuery expects the columns and types to align. In practice, I handle this in two ways.

First, I try to validate and clean the schema inside the pipeline itself. For example, I might cast a string date into a proper timestamp or drop extra fields that don't exist in the target table.

Second, if the change is unexpected like a new column appears I send the file to an error bucket or a dead-letter path, so it doesn't break the production pipeline. From there, I can investigate, adjust the schema, and replay the file.

In some projects, I've also used staging tables. The raw file loads into a staging table with a flexible schema, and then I use SQL in BigQuery to map it into the final schema. That gives more control when schema evolution is frequent.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

40. How do you configure Data Fusion to write processed data back into GCS?

To write data back into GCS, I just add a Cloud Storage sink at the end of my pipeline. In the configuration, I set the bucket name, the folder path, and the format I want like CSV, JSON, Avro, or Parquet.

For example, if I'm preparing files for analytics, I often choose Parquet or Avro because they're compressed and efficient. If I'm exchanging files with a partner, I might use CSV since it's simple.

I can also configure details like whether to include headers, how to compress the files, and how big each file should be. Once the pipeline runs, Data Fusion automatically writes the processed data into that GCS location. In one of my projects, we used this setup to create daily partitioned folders in GCS for downstream reporting.

41. Why would you use Dataproc clusters for Data Fusion pipelines?

In Data Fusion, I would use Dataproc clusters mainly when my pipeline involves large-scale data processing or heavy transformations. The default Data Fusion engine can handle normal data movements and lightweight transformations, but for big data use cases it's not efficient. Dataproc gives me the power of Spark and Hadoop, which can distribute the workload across multiple nodes. That way, I can process terabytes of data much faster. Another reason is cost and flexibility – with Dataproc I can create ephemeral clusters on demand, so I only pay when I run the job, and the cluster shuts down afterward. This helps when I need scalable compute power for big jobs but don't want to keep resources running all the time.

42. What's the difference between Spark runtime and Direct engine in Data Fusion?

The Direct engine in Data Fusion is used mainly for development and testing. It runs pipelines locally inside Data Fusion itself, without needing external resources like Spark. It is fast to test small amounts of data because I don't have to spin up a cluster. But it's not meant for production or heavy workloads, because it doesn't scale well.

The Spark runtime is used when the pipeline needs to handle big volumes of data or complex transformations. In this case, Data Fusion submits the job to a Dataproc cluster running Spark. Spark breaks the work into distributed tasks, which makes it scalable and much more efficient for production pipelines. So in short, Direct engine is good for quick testing and debugging, while Spark runtime is for scalable, production-grade processing.

43. When should you offload heavy transformations to Dataproc instead of running in Data Fusion?

I would offload heavy transformations to Dataproc whenever the operations involve large datasets, complex joins, aggregations, or machine learning tasks. Data Fusion itself is more about building and orchestrating the pipeline – it's great for ingestion, integration, and basic transformations. But if I try to run something like joining multiple huge tables or running iterative computations inside Data Fusion, it can become slow and costly.

By pushing these transformations to Dataproc, I can take advantage of Spark's distributed computing. For example, if I have to clean and aggregate logs that are hundreds of GBs, Spark will parallelize the work across many nodes, which reduces runtime significantly. Also, for transformations that need advanced libraries or custom Spark code, Dataproc is better suited. In short, whenever performance, scalability, or advanced computation is needed, I would offload the transformations to Dataproc instead of keeping them inside Data Fusion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

44. How do you tune performance when running Data Fusion pipelines on Dataproc?

When I run Data Fusion pipelines on Dataproc, tuning performance is about balancing cluster resources and optimizing how Spark jobs execute. First, I look at the cluster size. If the data is very large, I increase the number of worker nodes and adjust memory/CPU per node so Spark executors have enough resources. I also configure auto-scaling, so the cluster can scale up during heavy workloads and scale down when idle.

Second, I tune Spark configurations. For example, I adjust parameters like executor memory, number of cores, and parallelism so tasks don't fail because of lack of memory or get stuck due to too few partitions. I also make sure my input data is partitioned well in storage (like GCS or BigQuery) so Spark can process it in parallel without skew.

Third, I optimize pipeline design. Instead of chaining too many heavy transformations in Data Fusion, I try to push them efficiently into Spark transformations. For joins, I use broadcast joins if one dataset is small, which reduces shuffle overhead. Finally, I monitor Spark UI from Dataproc to identify bottlenecks, like skewed tasks or long-running stages, and then adjust partitioning or resources accordingly.

45. How can Data Fusion help migrate legacy Spark/Hive workloads to GCP?

Data Fusion is very useful when modernizing old Spark or Hive workloads because it provides a no-code/low-code interface to rebuild those pipelines on GCP. Many organizations already have Hive queries and Spark jobs running on on-prem Hadoop clusters, but moving them directly to the cloud can be complex.

With Data Fusion, I can connect to the same sources, replicate the transformations visually, and then run them on a Dataproc cluster. This way, I don't have to rewrite everything from scratch. For Hive queries, I can use the Wrangler or SQL transform plugins to replicate the logic. For Spark jobs, Data Fusion lets me offload heavy transformations to Dataproc, so it works very similar to on-prem Spark but with cloud benefits like auto-scaling and ephemeral clusters.

Also, Data Fusion helps standardize workflows. Instead of managing scripts manually, I can centralize the logic inside pipelines that are easy to monitor, version, and schedule. So the migration becomes not only a lift-and-shift but also an opportunity to simplify and modernize.

46. How can you use Cloud SQL as a source in Data Fusion?

To use Cloud SQL as a source in Data Fusion, I set up a connection through the Cloud SQL connector. Data Fusion provides ready-made plugins for relational databases, including Cloud SQL for MySQL, PostgreSQL, and SQL Server.

The first step is to make sure the Cloud SQL instance is accessible from Data Fusion. If my Data Fusion environment is private, I enable private IP for Cloud SQL. If not, I configure authorized networks or use the Cloud SQL Proxy. Then, inside Data Fusion, I create a connection by providing the JDBC URL, database name, username, and password (which I can store securely in Secret Manager).

Once the connection is set, I can use the Database Source plugin in my pipeline to read tables or run custom SQL queries from Cloud SQL. The data then flows into the pipeline and can be transformed or moved to targets like BigQuery, GCS, or even another database. This makes Cloud SQL act just like any other source system, fully integrated into my ETL/ELT pipelines.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

47. How can you configure Cloud Spanner as a sink in Data Fusion?

To configure Cloud Spanner as a sink, I first create a connection in Data Fusion using the Cloud Spanner plugin. I provide details like the Spanner instance ID, database name, project ID, and service account with the right IAM permissions. Once the connection is set, I can choose Cloud Spanner Sink in the pipeline. In the sink configuration, I specify the target table, schema mapping, and write disposition (append, update, or overwrite depending on use case).

If the pipeline's schema doesn't perfectly match Spanner's table schema, I make sure to map columns correctly or use transformations in Data Fusion to align them. I also take care with primary keys, since Spanner requires them. After setting up, the pipeline can load processed data into Spanner tables at scale, and I can use it for transactional or analytical workloads.

48. How is Bigtable integrated with Data Fusion pipelines?

Bigtable integration in Data Fusion is through Bigtable source and sink connectors. If I want to read from Bigtable, I configure a source connection by giving project ID, instance ID, table name, and optional row key ranges or filters. This allows Data Fusion to pull data from Bigtable into the pipeline.

As a sink, Data Fusion can write the processed data into Bigtable tables. I configure the sink with similar details and specify how row keys should be created, since Bigtable performance depends heavily on row key design. This makes it possible to use Data Fusion to feed cleaned, structured, or aggregated data into Bigtable for serving low-latency queries.

49. How can Data Fusion use Cloud DLP to mask sensitive fields in data?

Data Fusion integrates with Cloud DLP using the Cloud DLP plugin. In a pipeline, I can add the DLP transform stage where I configure which fields need masking or tokenization. For example, if I have columns like credit card number, email, or phone number, I can select those and apply DLP transformation.

Cloud DLP then uses its built-in de-identification techniques, like redaction, tokenization, or hashing, and returns the safe version of the data. This way, my pipeline continues with masked data while ensuring compliance with privacy rules. This is especially useful if I'm moving data into environments like analytics or reporting, where sensitive details don't need to be exposed.

50. How can Looker Studio consume data prepared by Data Fusion pipelines?

Looker Studio doesn't directly connect to Data Fusion, but it connects very well to the targets where Data Fusion writes data. Normally, Data Fusion pipelines output processed data into BigQuery, Cloud SQL, or even Cloud Spanner. Looker Studio has native connectors for these sources.

So once the pipeline loads the cleaned and transformed data into BigQuery, I can create a Looker Studio report by simply linking it to that BigQuery dataset. The pipeline ensures the data is refreshed and reliable, while Looker Studio provides the visualization layer. In this way, Data Fusion acts as the ETL backbone, and Looker Studio is the front-end for dashboards and insights.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

51. How are failed records captured in Data Fusion pipelines?

In Data Fusion, failed records are captured through the error handling mechanism. Most source, transform, and sink plugins provide an "error port." If a record cannot be processed, maybe due to schema mismatch, type conversion error, or invalid data format, that record is sent to the error port instead of breaking the whole pipeline. I can connect this error port to another sink, like Cloud Storage, BigQuery, or Pub/Sub, to capture and store failed records.

This way, the pipeline continues to run without stopping, and I can review the failed records later for debugging or reprocessing. For example, if I'm loading customer data into BigQuery and some rows have invalid dates, those rows get sent to an error bucket in GCS while the rest of the data still lands in BigQuery successfully.

52. What is a dead-letter queue and how is it used in streaming pipelines?

A dead-letter queue (DLQ) is a storage location, usually a Pub/Sub topic, where messages that could not be processed successfully are sent. In streaming pipelines, data is continuous, and a single bad record can't be allowed to block the stream. So instead of dropping the record or failing the whole job, the pipeline routes those problematic events to the DLQ.

This allows me to keep the streaming pipeline running smoothly while also not losing any data. Later, I can investigate the DLQ to understand why records failed and either fix them manually or reprocess them with a correction pipeline. For example, if I'm streaming IoT sensor data into BigQuery and some messages arrive in the wrong format, those invalid messages go into the DLQ Pub/Sub topic for review, while the rest of the stream keeps flowing.

53. How does Data Fusion retry failed jobs automatically?

Data Fusion has a built-in retry mechanism for jobs. If a pipeline run fails because of transient issues, like network glitches or temporary unavailability of a service, Data Fusion can be configured to retry the job automatically. In the pipeline configuration, I can set the number of retries and the delay between retries.

For batch pipelines, this means if the first run fails, Data Fusion waits for the configured delay and then restarts the run up to the retry limit. For streaming pipelines, Data Fusion restarts the pipeline automatically when it fails. This ensures high availability and reduces manual intervention. For example, if I'm running a pipeline that writes to BigQuery and it fails due to a temporary quota issue, retries allow it to succeed on the next attempt without me restarting it manually.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

54. How can you debug failed transformations using error logs?

When a transformation fails in Data Fusion, the first step I take is to check the pipeline run logs. Each pipeline stage has detailed logs that show why a record or transformation failed. For example, the log might point out a schema mismatch, a null value in a non-null column, or a parsing error.

If I enable error ports, I can also capture the failed records separately in a BigQuery table or Cloud Storage bucket. By reviewing both the logs and the failed records, I can understand whether the issue is due to bad input data, incorrect transformation logic, or configuration mistakes.

For more complex debugging, I sometimes use the Dataproc Spark UI if the pipeline was running on Spark, because it gives deeper insights into task failures and execution bottlenecks. Overall, logs and error records together give me a clear picture to fix the transformation logic and rerun successfully.

55. What are some strategies for building fault-tolerant pipelines in Data Fusion?

To build fault-tolerant pipelines, I focus on designing them so that failures don't stop the entire process. Some strategies I use are:

1. Enabling error ports on transforms and sinks, so bad records are captured without breaking the pipeline.
2. Using dead-letter queues in streaming pipelines to handle problematic messages while letting the rest flow.
3. Configuring retries and backoff policies, so temporary failures like network issues get resolved automatically.
4. Breaking pipelines into smaller, modular stages instead of one big flow, so failures can be isolated.
5. Designing idempotent pipelines, meaning if a job is retried, it won't duplicate or corrupt the data.
6. Adding monitoring and alerts through Cloud Monitoring, so I know quickly when something fails.

This way, the pipeline keeps running even if some data fails, and I can recover gracefully without major downtime.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

56. What are the main IAM roles available in Data Fusion?

In Data Fusion, IAM roles control what level of access a user has. The main predefined roles are:

1. Viewer – Can only view Data Fusion instances and pipelines but cannot edit or run them. Useful for read-only access.
2. Editor – Can create, modify, and run pipelines. This is the role most data engineers use for building and managing pipelines.
3. Admin – Full control over Data Fusion instances and configurations, including managing permissions and connections.
4. Developer (specific to Data Fusion) – Can create and manage pipelines inside the environment but does not have admin-level permissions over the instance itself.

In practice, I assign roles based on least privilege. For example, analysts might get Viewer, engineers get Developer or Editor, and only a few team members get Admin for governance.

57. Why are service accounts important in Data Fusion pipelines?

Service accounts are very important in Data Fusion pipelines because they define what resources the pipeline can access. Each pipeline stage, like reading from GCS or writing to BigQuery, requires specific permissions. Instead of using personal credentials, I attach a service account with the right IAM roles to the pipeline. This makes access secure, controlled, and auditable.

For example, if my pipeline needs to read from Cloud SQL and write to BigQuery, I would create a service account with cloudsql.client and bigquery.dataEditor roles, and then configure my pipeline to run under that service account. This way, the pipeline only has the exact permissions it needs, nothing more, following the principle of least privilege.

58. How does Data Fusion provide data lineage visualization?

Data Fusion automatically builds data lineage as I design and run pipelines. The lineage view shows how data flows from source to sink, including all transformations in between. This helps me understand where each field comes from, what changes were applied, and where it finally lands.

For example, if I'm pulling sales data from Cloud SQL, applying transformations, and writing to BigQuery, the lineage graph will show the source table, the transform steps, and the target dataset. This is very useful for debugging, auditing, and explaining the pipeline to business teams. It also helps meet compliance needs, because I can trace exactly how sensitive fields were handled.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

59. How does Data Fusion support metadata management for compliance?

Data Fusion integrates with the Data Catalog service in GCP, which allows me to manage metadata about datasets. When a pipeline writes to BigQuery or other sinks, Data Fusion can push metadata like schema, column names, and lineage into Data Catalog. This means I can centrally search and discover datasets, track schema versions, and apply tags like PII classification.

For compliance, this metadata is very important. It lets organizations demonstrate where sensitive data is stored, how it flows across systems, and who has access. For example, if I need to comply with GDPR or HIPAA, I can use Data Catalog tags to label sensitive columns and make sure policies are enforced.

60. What encryption options are supported in Data Fusion (CMEK vs Google-managed keys)?

Data Fusion supports two main types of encryption:

1. Google-managed keys – This is the default option, where Google manages the encryption keys for you. All data at rest and in transit is automatically encrypted without me doing any extra configuration.
2. Customer-managed encryption keys (CMEK) – Here, I create and manage my own keys in Cloud KMS. I can assign these keys to my Data Fusion instance, pipelines, and target storage. This gives me more control, especially if my organization has strict compliance requirements. For example, I can rotate keys regularly, revoke access if needed, and prove full control over data encryption.

In practice, if compliance is strict, I go with CMEK; if not, Google-managed keys are simpler and sufficient.

61. How can you improve pipeline performance using parallelism in Data Fusion?

To improve performance with parallelism, I design my pipeline so that tasks can run concurrently instead of sequentially. For example, if I have multiple independent branches of data processing, I connect them in parallel stages rather than chaining them. This allows Data Fusion to process more data at the same time.

Also, I take advantage of Spark runtime parallelism when using Dataproc. By increasing the number of partitions in the source data and adjusting Spark's parallelism settings, I make sure the workload is spread evenly across executors. For example, reading a large table from BigQuery with sufficient partitions will let Spark handle multiple chunks in parallel, which speeds up the pipeline significantly.

So, parallelism in Data Fusion is about both pipeline design and configuring the underlying engine to split and distribute the work efficiently.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

62. What role does cluster sizing play when using Dataproc runtime?

Cluster sizing is critical because it directly affects performance and cost. If I size the cluster too small, my Spark jobs will run slowly, tasks may fail due to lack of memory, and the pipeline could become a bottleneck. If I size it too large, I might waste resources and increase costs unnecessarily.

The right sizing depends on the data volume and complexity of transformations. For example, if I'm processing 10 GB of data, a small 2–3 node cluster may be fine. But for terabytes of data with heavy joins and aggregations, I need a larger cluster with more workers and memory per executor.

Dataproc also supports autoscaling, which is very helpful. I usually configure autoscaling so the cluster grows during peak load and shrinks when idle, giving me a good balance of performance and cost.

63. Why should you avoid small files in GCS when designing Data Fusion pipelines?

Small files are a common performance issue in big data pipelines. Each file, no matter how small, requires overhead for opening, reading, and metadata operations. If I have thousands of tiny files in GCS, Spark or Data Fusion spends more time managing files than actually processing data, which slows everything down.

For example, if I receive log data every minute and store each log as a separate small file, the pipeline might choke with too many file handles even if the total data size is not huge. The best practice is to batch or merge small files into larger ones, like 100 MB to 1 GB, before processing. This way, Spark can process fewer, bigger files more efficiently with less overhead.

In short, avoiding small files helps reduce overhead, improves parallelism, and speeds up the overall pipeline execution.

64. How can schema design impact pipeline performance?

Schema design has a direct effect on performance because it determines how efficiently data can be read, transformed, and written. If the schema has too many nested or complex fields, transformations become slower since the pipeline needs extra steps to parse and flatten them. Wide schemas with hundreds of columns can also increase memory usage and shuffle overhead in Spark.

Another issue is data types. If the schema uses inconsistent or suboptimal data types (like storing numbers as strings), every transformation requires extra type conversion, which slows things down. Similarly, missing or constantly changing schemas cause frequent pipeline errors that add retries and delays.

On the other hand, a well-designed schemawhere fields are clean, typed correctly, and structured in a way that matches the target systemreduces transformations and improves efficiency. For example, if I know BigQuery is the target, I design the schema to align with BigQuery's column types upfront, so the pipeline can load data with minimal conversions.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

65. When would you choose a batch pipeline instead of streaming for performance reasons?

I would choose batch pipelines when the data does not need to be processed in real time and can be grouped together to reduce overhead. In streaming, each record or small micro-batch is processed continuously, which puts more load on the system. If I'm dealing with huge volumes of data that arrive at regular intervals, processing them as a batch can be much faster and more cost-efficient.

For example, if I get sales transaction data from Cloud SQL every night, running a batch pipeline to process millions of rows at once is more efficient than trying to stream every single record during the day. Batch jobs also allow me to optimize transformations, apply large joins, and leverage Spark's parallelism better than handling small streaming events.

So, when low latency is not a requirement but throughput and efficiency matter, batch pipelines are the better choice.

66. How would you design a CDC pipeline from Cloud SQL into BigQuery using Data Fusion?

To design a CDC (Change Data Capture) pipeline from Cloud SQL to BigQuery, I would first enable the Datastream service to capture changes from Cloud SQL in real time. Datastream streams inserts, updates, and deletes into GCS in the form of change logs.

Then, in Data Fusion, I build a pipeline that picks up these change files from GCS. I add transformations if needed, like mapping Cloud SQL schema to BigQuery schema or masking sensitive data. Finally, I configure BigQuery as the sink and enable merge or upsert logic so that changes are applied correctly to the target table.

For example, if a row is updated in Cloud SQL, Datastream captures the change, Data Fusion processes it, and BigQuery reflects the update instead of creating duplicates. This ensures the BigQuery table always stays in sync with Cloud SQL.

This approach provides a near-real-time CDC pipeline that is reliable, scalable, and easy to monitor.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

67. How would you build a streaming ETL pipeline from Pub/Sub to BigQuery with transformations?

I'd explain my approach in three parts: design, implementation, and operational concerns.

Design

- Start by defining requirements: latency (near-real time vs seconds), expected throughput, exactly-once vs at-least-once semantics, schema evolution, and SLAs for freshness.
- Choose Pub/Sub as the ingestion layer (publisher → topic → subscription). BigQuery will be the sink (streaming inserts or micro-batch loads).
- Decide where transformations happen: use Dataflow/Cloud Data Fusion (Dataproc Spark runtime) depending on complexity. For continuous transformations with windowing and event-time semantics I prefer Dataflow; for visual pipelines with simpler transforms Data Fusion + Dataproc can work.

Implementation (concrete steps)

1. Pub/Sub setup
    - Create topic(s) and durable subscription(s). Use separate topics for different event types if schema differs.
    - Publishers must include a schema/version or embed a schema version id in message attributes for evolution handling.
2. Ingest & pre-processing
    - Build a streaming pipeline (Dataflow streaming template or Data Fusion streaming pipeline). The pipeline consumes Pub/Sub subscription.
    - Validate and parse messages immediately (JSON -> typed record). Send invalid messages to a dead-letter Pub/Sub topic or GCS for later inspection.
3. Transformations
    - Apply stateless transforms: parse, field renames, type conversions, DLP masking/tokenization for PII.
    - Apply stateful transforms when needed: aggregation, session/windowed metrics, deduplication using event IDs and event time. Use Watermarks and windowing (if using Dataflow) to handle late data.
    - For joins with reference data, cache small reference sets in memory (side input) or read periodically from BigQuery/Firestore. For large joins, use an enriching lookup service or offline join approach.
4. Write to BigQuery
    - Use streaming inserts to a partitioned & clustered BigQuery table if near-real-time is required. Partition by event_date (or ingestion_timestamp) and cluster on high-selectivity columns.
    - For high-throughput pipelines prefer micro-batch writes to GCS (Avro/Parquet) and load jobs into BigQuery to avoid streaming quota limits and reduce cost. Automate micro-batch windows (e.g., every 1-5 minutes).
    - Use insertId or deterministic deduplication logic to achieve exactly-once semantics in BigQuery when necessary.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. Error handling & DLQ
    - Route malformed records or transformation errors to a DLQ (Pub/Sub or GCS), capture metadata (error reason, original payload, timestamp) for reprocessing.

Operational considerations

- Monitoring: create Cloud Monitoring dashboards for Pub/Sub backlog, Dataflow job metrics (latency, watermarks), BigQuery streaming insert errors, and DLQ size. Alert on growing backlog or high error rates.
- Scalability: Autoscaling for Dataflow or Dataproc; tune parallelism and worker type based on throughput.
- Idempotency: design sinks/updates to be idempotent (use unique keys or merge/upsert logic).
- Schema evolution: use schema registry or include versioning; handle additive changes gracefully.
- Security/compliance: use CMEK if needed, DLP for masking, least-privilege service accounts.

One-line interview wrap-up: I'd pick streaming tech that supports event-time semantics (Dataflow for complex streaming), validate early, route bad messages to a DLQ, write to partitioned BigQuery (stream or micro-batch), and monitor/backpressure closely.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

68. How do you design a batch ingestion pipeline from GCS into BigQuery using Data Fusion?

I'd design it around reliability, cost-efficiency, and schema alignment. Steps I'd take:

Requirements & planning

- Confirm input file formats (CSV/JSON/Avro/Parquet), file sizes, frequency, schema, desired freshness, and row-level deduplication/upsert needs.
- Decide on load strategy: direct BigQuery load jobs (recommended for large files / cost-efficiency) vs streaming inserts (for small frequent updates).

Pipeline design (step-by-step)

1. Landing & organization
    - Land files in GCS with clear prefix/date partitioning (e.g., gs://bucket/data/yyyy=2025/mm=09/dd=11/). Avoid small files prefer batching into larger files.
    - Use object naming or a manifest to identify which files belong to a run.
2. Data Fusion pipeline
    - Source: GCS File source (read Avro/Parquet/CSV).
    - Validation & cleansing: apply Wrangler or transform plugins to drop bad rows, fix types, normalize fields, apply DLP if needed. Send unfixable rows to an error sink (GCS or BigQuery error table).
    - Schema mapping: map fields to BigQuery schema, cast types, and handle nullability/fields missing.
    - Deduplication/upsert: if required, add keys and use staging table + MERGE into production table. For example: write cleansed files into a staging BigQuery table and run a MERGE SQL to upsert into final table.
3. Load strategy to BigQuery
    - Preferred: write cleaned data to GCS as Avro/Parquet and invoke a BigQuery load job (batch) this is cheaper and avoids streaming quotas. Use partitioned tables and set writeDisposition (WRITE_APPEND or WRITE_TRUNCATE) as per needs.
    - Alternative: directly use BigQuery sink from Data Fusion if dataset is small or if simpler flow is acceptable.
4. Orchestration & scheduling
    - Use Cloud Composer (Airflow) or Data Fusion schedules to run the pipeline after file arrival; optionally trigger via Pub/Sub notifications for the bucket.
    - Maintain idempotency: mark processed files (move to archive/processed prefix) or use lock/manifest to avoid double processing.
5. Monitoring & retries
    - Capture pipeline job status, failed rows, and load job errors. Configure retries with exponential backoff. Alert on failures or skewed runtimes.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Best practices & optimizations

- Use partitioned & clustered BigQuery tables; load into partitions directly.
- Avoid small files; merge into optimal sizes (100MB–1GB) before load.
- Use Avro/Parquet for schema preservation and faster loads.
- Use service accounts with least privilege and secure credentials in Secret Manager.

One-line interview wrap-up: I'd land files in GCS, clean/validate in Data Fusion, write optimized Avro/Parquet, run BigQuery load jobs into partitioned/clustered tables, and use orchestration and DLQs for reliability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

69. What steps would you include in a data quality pipeline for cleansing and validation?

A good data quality pipeline is proactive and layered. I'd structure it as validation → cleansing → enrichment → monitoring, with visibility and reprocessing hooks.

Key steps

1. Schema validation

   Check required fields, data types, and field-level constraints. Reject or send non-conforming records to an error store with reason codes.

2. Row-level validation

   Business rules: e.g., order_date <= ship_date, amount >= 0, valid enumerations. Tag or route failing rows to DLQ.

3. Null and default handling

   Define allowed nulls; for missing optional fields, fill sensible defaults; for required fields, decide to reject or enrich from reference data.

4. Type normalization & conversion

   Convert strings to numeric/date types with safe parsing (log parse failures), trim whitespace, unify case for categorical fields.

5. Deduplication

   Use deterministic keys or fingerprinting (hash of canonical fields) to identify duplicates. For streaming, keep a short-lived deduplication store; for batch use staging + dedupe prior to final load.

6. Referential integrity & enrichment

   Validate foreign keys against reference datasets. Enrich records by joining with master data (small lookup caches or periodic batch joins).

7. Field-level masking & PII detection

   Discover PII (or use Data Catalog tags) and apply masking/tokenization via Cloud DLP or built-in transforms.

8. Validation metrics & score

   Emit data quality metrics: row counts, fail rates, percent nulls, cardinality checks. Compute a quality score and store it (BigQuery/Monitoring).

9. Error capture & reprocessing

   Route failed records to DLQ with metadata. Provide a reprocessing pipeline after fixes, with traceability to original batch/run ID.

10. Automated alerts & SLAs

    Alert when thresholds breached (e.g., >1% error rate), or when expected daily counts deviate.

Tooling & operational tips

- Implement assertions early (ingest layer) to fail fast.
- Keep transformations idempotent to support retries.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

- Store lineage and provenance so business users can trace issues.
- Use incremental checks (row count, min/max) to detect anomalies quickly.

One-line interview wrap-up: I'd validate schema and business rules first, cleanse and enrich, deduplicate, mask PII, send failures to a DLQ, and continuously measure quality with alerts and reprocessing paths.

70. What are the main considerations when migrating ETL jobs from Informatica to Data Fusion?

Migrating ETL from Informatica to Data Fusion is a lift-and-modernize project. I'd focus on functional parity, performance, security, and operationalization.

Key considerations

1. Discovery & inventory

   Catalog all Informatica mappings, workflows, schedules, connections, custom scripts, and dependencies. Identify which jobs are critical and the data volumes they handle.

2. Functional mapping

   Map Informatica transforms to Data Fusion plugins or custom transforms. Simple tasks (filter, join, aggregate) map easily; complex or proprietary transforms may need custom Spark code or Dataproc jobs.

3. Connection & connectivity

   Recreate source/target connections (databases, files, REST APIs). Ensure network connectivity (VPC, Private IP, Cloud SQL proxy) and service account permissions are set.

4. Performance & architecture

   Analyze current performance. For heavy workloads, plan Dataproc-backed Spark runtime and right cluster sizing. Consider partitioning and storage formats (Parquet/Avro) to improve throughput.

5. State & metadata

   Migrate metadata, table schemas, transformation logic, and parameterization. Integrate with Data Catalog for discoverability and maintain lineage.

6. Error handling & DLQs

   Recreate error handling: error ports, DLQ topics, retry logic. Ensure equivalent handling of failed records.

7. Scheduling & orchestration

   Replace Informatica scheduler with Cloud Composer (Airflow) or Data Fusion schedules. Rebuild dependency graphs, SLA checks, and alerts.

8. Testing & validation

   Implement parallel runs: run Informatica and Data Fusion in parallel, compare outputs row-by-row and metrics. Validate edge cases, performance, and data quality.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

9. Security & compliance

   Reapply least-privilege IAM, secrets management, CMEK if needed, and DLP masking. Validate audit logs and access controls.

10. Team skills & change management

    Train teams on Data Fusion, Dataflow, and Dataproc. Update runbooks and incident procedures.

11. Cost & operational model

    Estimate costs (Dataproc, storage, BigQuery). Use autoscaling and ephemeral clusters to reduce spend. Build monitoring and SLO-driven ops.

12. Phased migration approach

    Migrate low-risk jobs first, then medium, then critical. Use this to refine patterns and templates for remaining conversions.

One-line interview wrap-up: I'd do a full discovery, map transforms to Data Fusion/Dataproc equivalents, validate outputs with parallel runs, re-establish scheduling and DLQs, and optimize for cloud cost and performance while maintaining security and lineage.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar